

山东大学



网络安全创新创业实践

作业 4: SM3 软件实现与优化

姓名	学号	班级
钟晴	202322460127	密码 2 班
宋坤鹏	202322460130	密码 1 班
赵思鹏	202300460075	密码 1 班
田学琛	202300460064	密码 1 班

目录

1	实验要求	1
1.1	作业题目	1
1.2	实验目的	1
1.3	实验环境	1
2	实验原理	2
2.1	SM3 算法	2
2.2	SIMD 寄存器与通用寄存器混合优化原理	7
2.3	x86 架构下的 AVX2 与 AVX-512 原理	8
2.4	ARM64 架构下的 NEON 指令集原理	9
3	实验设计与验证方法	10
3.1	总体实验设计	10
3.2	正确性测试设计	11
3.3	性能测试方法	12
4	x86 AVX2 实现与实验	13
4.1	AVX2 八路实现	13
4.2	x86 实验流程	14
4.3	x86 正确性与指令验证	14
4.4	x86 性能结果	16
5	ARM64 NEON 实现与实验	18
5.1	NEON 四路实现	18
5.2	ARM64 实验流程	18
5.3	ARM64 正确性与指令验证	20
5.4	ARM64 性能结果	22
6	实验结果分析	23
6.1	x86 与 ARM64 结果对比	23
6.2	实验中遇到的问题	24
6.3	进一步优化方向	25
7	结论	25

1 实验要求

1.1 作业题目

作业 4: SM3 软件实现与优化。

实现基于 SIMD 寄存器和通用寄存器混合的 SM3 算法优化实现, 要求实现 ARM64 和 x86 (AVX2 或 AVX-512) 两种架构指令集。

根据题目要求, 本实验从 SM3 标量参考实现出发, 分别完成 x86 AVX2 八路并行实现和 ARM64 NEON 四路并行实现。程序使用 SIMD 寄存器并行执行消息扩展、布尔函数、模加和循环移位等规则运算, 同时使用通用寄存器完成消息指针、消息长度、分组编号、循环控制和活跃 lane 管理。

1.2 实验目的

本实验的主要目的如下:

- (1) 掌握 SM3 算法的基本原理, 理解消息填充、消息分组、消息扩展、压缩函数和链值更新等主要步骤;
- (2) 掌握 x86 AVX2 指令集及 YMM 向量寄存器的使用方法, 实现一次并行处理 8 条消息的 SM3 八路优化后端;
- (3) 掌握 ARM64 NEON 指令集及 V 向量寄存器的使用方法, 实现一次并行处理 4 条消息的 SM3 四路优化后端;
- (4) 设计 SIMD 寄存器与通用寄存器混合的实现结构, 使 SIMD 寄存器负责规则的向量运算, 通用寄存器负责地址计算、循环控制、消息长度和活跃 lane 管理;
- (5) 通过标准向量、随机差分、OpenSSL 对比、反汇编和性能测试, 验证实现的正确性、目标指令生成情况及优化效果。

1.3 实验环境

本实验包含 x86-64 与 ARM64 两种架构。两种平台均采用 GCC 进行优化编译, 以标量 SM3 作为参考后端, 以 SIMD 多缓冲区实现作为优化后端。

表 1: 实验环境

项目	x86-64 平台	ARM64 平台
处理器架构	x86-64, 支持 AVX2, 不支持 AVX-512F	AArch64, 阿里云倚天 710
运行环境	Linux/WSL2	阿里云 ECS ecs.c8y.large
CPU 与内存	本机处理器	2 vCPU、4 GiB
操作系统	Linux	Ubuntu 22.04 LTS ARM64
SIMD 指令集	AVX2	NEON/Advanced SIMD
编译选项	-O3 -mavx2	-O3 -march=armv8-a+simd
正确性参考	标量实现	标量实现、OpenSSL SM3

x86 平台通过 `lscpu` 检测到 AVX2, 但未检测到 AVX-512F, 因此选择 AVX2 八路实现作为主要 x86 后端。ARM64 云服务器执行得到 `aarch64`, 处理器 Features 中包含 `asimd`, 说明平台支持 ARM64 Advanced SIMD, 即 NEON。

```
Linux satt12 6.8.0-124-generic #124-22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Tue May 26 21:05:19 UTC x86_64 x86_64 x86_64
GNU/Linux
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 45 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 2
On-line CPU(s) list: 0,1
Vendor ID: AuthenticAMD
Model name: AMD Ryzen 5 5600U with Radeon Graphics
CPU family: 25
Model: 80
Thread(s) per core: 1
Core(s) per socket: 1
Socket(s): 2
Stepping: 0
BogoMIPS: 4591.20
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep ntrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 syscall nx mmxext pdpeibg rdtscp l
n constant_tsc rep_good nopl tsc_reliable nonstop_tsc cpuid extd_a
picld tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 x2
apic movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cr8
legacy_ahb sse4a misalignsse 3dnowprefetch osvw topext lbpl vmc
all fsgsbase bmi1 avx2 smap bmi2 erms rdseed adx smap clflushopt c
lwb sha_ni xsaveopt xsavec xgetbv1 xsaves clzero arat umip vaes vp
clmulqdq rdpid overflow_recov succor fsrn
Virtualization features:
Hypervisor vendor: VMware
Virtualization types: full
Caches (sum of all):
L1d: 64 KIB (2 instances)
L1i: 64 KIB (2 instances)
L2: 1 MiB (2 instances)
L3: 32 MiB (2 instances)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0,1
```

方案一

推荐方案

ARM开发测试

倚天710架构, 2核4G轻量计算, 性价比高

¥0.308933/时

配置包含 [修改](#)

- 实例规格 计算型 c8y ecs.c8y.large (2 vCPU 4 GiB)
- 操作系统 Ubuntu 22.04 64位 ARM版
- 系统盘 ESSD云盘 40 GiB PL0
- 数据盘 不含
- 公网带宽 按使用流量 5Mbps
- 安全组开放端口 22/22

(a) x86 实验平台配置 (b) ARM64 云服务器配置

图 1: x86 与 ARM64 实验平台配置

2 实验原理

2.1 SM3 算法

SM3 是一种迭代型密码杂凑算法, 能够将任意长度的输入消息映射为固定长度的 256 bit 摘要。算法整体采用逐分组迭代结构, 主要包括消息预处理、消息扩展、压缩函数迭代和摘要输出。

设填充后的消息包含 n 个 512 bit 分组, 记为

$$B^{(0)}, B^{(1)}, \dots, B^{(n-1)}.$$

(1)

算法从固定初始向量 $V^{(0)}$ 出发, 依次调用压缩函数处理各消息分组:

$$V^{(i+1)} = \text{CF}(V^{(i)}, B^{(i)}), \quad 0 \leq i < n. \quad (2)$$

全部分组处理完成后, 最终链值 $V^{(n)}$ 即为消息的 SM3 摘要。

2.1.1 消息预处理

设原始消息为 m , 长度为 l bit, 并要求 $l < 2^{64}$ 。首先在消息末尾添加一个比特 1, 再添加 k 个比特 0, 使得

$$l + 1 + k \equiv 448 \pmod{512}. \quad (3)$$

随后附加原始消息长度 l 的 64 bit 大端编码。填充后的消息可表示为

$$m' = m \| 1 \| 0^k \| [l]_{64}, \quad (4)$$

其中, $\|$ 表示比特串连接, $[l]_{64}$ 表示消息长度 l 的 64 bit 二进制表示。此时填充后的总长度满足

$$l + 1 + k + 64 \equiv 0 \pmod{512}. \quad (5)$$

填充后的消息按照顺序划分为 n 个 512 bit 分组:

$$m' = B^{(0)} \| B^{(1)} \| \dots \| B^{(n-1)}. \quad (6)$$

每个分组再按照大端序解析为 16 个 32 bit 字:

$$B^{(i)} = W_0 \| W_1 \| \dots \| W_{15}. \quad (7)$$

对于连续四个输入字节 b_{4j} 、 b_{4j+1} 、 b_{4j+2} 和 b_{4j+3} , 对应的 32 bit 字为

$$W_j = (b_{4j} \ll 24) | (b_{4j+1} \ll 16) | (b_{4j+2} \ll 8) | b_{4j+3}. \quad (8)$$

SM3 使用大端序解释输入数据, 因此在常见的小端处理器上, 需要在装载消息字时进行显式字节序转换。

消息填充还存在重要边界情况。一个 512 bit 分组共有 64 字节, 其中最后 8 字节用于保存消息长度。若原始消息长度为 55 字节, 当前分组仍能容纳填充标志和长度字段; 若长度达到 56 字节, 则必须增加一个新的消息分组。因此, 55/56、63/64 和 119/120 字节等边界需要在正确性测试中重点覆盖。

2.1.2 初始参数与基本函数

SM3 使用固定的 256 bit 初始向量 $V^{(0)}$ 。将其划分为 8 个 32 bit 字, 有

$$V^{(0)} = \{7380166F, 4914B2B9, 172442D7, DA8A0600, \\ A96F30BC, 163138AA, E38DEE4D, B0FB0E4E\}. \quad (9)$$

压缩函数共执行 64 轮, 每轮使用轮常量 T_j :

$$T_j = \begin{cases} 79CC4519, & 0 \leq j \leq 15, \\ 7A879D8A, & 16 \leq j \leq 63. \end{cases} \quad (10)$$

实际计算中使用循环左移后的轮常量

$$\text{ROTL}(T_j, j \bmod 32). \quad (11)$$

SM3 压缩函数还使用两个布尔函数 FF_j 和 GG_j 。对于前 16 轮, 有

$$FF_j(X, Y, Z) = X \oplus Y \oplus Z, \quad (12)$$

$$GG_j(X, Y, Z) = X \oplus Y \oplus Z. \quad (13)$$

对于后 48 轮, 有

$$FF_j(X, Y, Z) = (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z), \quad (14)$$

$$GG_j(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Z). \quad (15)$$

后 48 轮的 FF_j 属于 majority 函数, 其输出由三个输入中的多数比特决定; GG_j 属于 choose 函数, 由 X 决定选择 Y 或 Z 的对应比特。

此外, SM3 使用两个置换函数:

$$P_0(X) = X \oplus \text{ROTL}(X, 9) \oplus \text{ROTL}(X, 17), \quad (16)$$

$$P_1(X) = X \oplus \text{ROTL}(X, 15) \oplus \text{ROTL}(X, 23). \quad (17)$$

其中, P_0 用于压缩函数中的状态更新, P_1 用于消息扩展。循环移位和异或能够使输入字的不同位置相互混合, 从而增强算法的扩散性。

2.1.3 消息扩展与压缩迭代

每个 512 bit 消息分组最初只包含 16 个 32 bit 字。为了支持 64 轮压缩, 需要将其扩展为 68 个字:

$$W_0, W_1, \dots, W_{67}. \quad (18)$$

其中, W_0 至 W_{15} 直接来自消息分组。对于 $16 \leq j < 68$, 按照下式递推:

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus \text{ROTL}(W_{j-3}, 15)) \oplus \text{ROTL}(W_{j-13}, 7) \oplus W_{j-6}. \quad (19)$$

在此基础上, 再生成 64 个辅助扩展字:

$$W'_j = W_j \oplus W_{j+4}, \quad 0 \leq j < 64. \quad (20)$$

消息扩展使原始分组中的局部变化逐步传播到后续多个扩展字, 并影响更多压缩轮次。

对第 i 个消息分组进行压缩时, 首先将输入链值

$$V^{(i)} = (V_0^{(i)}, V_1^{(i)}, \dots, V_7^{(i)}) \quad (21)$$

分别赋值给八个工作变量:

$$\begin{aligned} A &= V_0^{(i)}, \quad B = V_1^{(i)}, \quad C = V_2^{(i)}, \quad D = V_3^{(i)}, \\ E &= V_4^{(i)}, \quad F = V_5^{(i)}, \quad G = V_6^{(i)}, \quad H = V_7^{(i)}. \end{aligned} \quad (22)$$

第 j 轮首先计算

$$A_{12} = \text{ROTL}(A, 12), \quad (23)$$

随后计算

$$SS1 = \text{ROTL}(A_{12} + E + \text{ROTL}(T_j, j \bmod 32), 7), \quad (24)$$

$$SS2 = SS1 \oplus A_{12}, \quad (25)$$

$$TT1 = FF_j(A, B, C) + D + SS2 + W'_j, \quad (26)$$

$$TT2 = GG_j(E, F, G) + H + SS1 + W_j. \quad (27)$$

上述所有加法均为模 2^{32} 加法, 即运算结果仅保留低 32 bit。完成中间量计算后, 工

作变量更新为

$$\begin{aligned}
 D &= C, \\
 C &= \text{ROTL}(B, 9), \\
 B &= A, \\
 A &= TT1, \\
 H &= G, \\
 G &= \text{ROTL}(F, 19), \\
 F &= E, \\
 E &= P_0(TT2).
 \end{aligned} \tag{28}$$

经过 64 轮迭代后, 将新的八个工作变量与当前输入链值逐字异或:

$$V^{(i+1)} = (A, B, C, D, E, F, G, H) \oplus V^{(i)}. \tag{29}$$

该结果作为下一个消息分组的输入链值。

2.1.4 摘要输出

全部 n 个消息分组处理完成后, 得到最终链值

$$V^{(n)} = (V_0^{(n)}, V_1^{(n)}, \dots, V_7^{(n)}). \tag{30}$$

将八个 32 bit 链值按照大端序依次连接:

$$\text{SM3}(m) = V_0^{(n)} \| V_1^{(n)} \| \dots \| V_7^{(n)}, \tag{31}$$

即可得到 256 bit, 即 32 字节的 SM3 消息摘要。

从软件实现角度看, SM3 具有以下特征:

- (1) 消息扩展和压缩函数均以 32 bit 无符号整数为基本单位, 主要包含模加、异或、与、或和循环移位;
- (2) 同一条消息的压缩轮次之间存在严格的数据依赖, 第 $j+1$ 轮必须等待第 j 轮完成, 因此难以直接进行轮级并行;
- (3) 多条独立消息之间不存在数据依赖, 可以将不同消息的相同状态字放入 SIMD 寄存器中, 采用多缓冲区方式并行处理;
- (4) 消息地址、消息长度、分组数量和填充位置可能不同, 需要使用通用寄存器完成地址计算、循环控制和消息调度;

- (5) 因此, SM3 适合采用“SIMD 寄存器负责规则数据运算, 通用寄存器负责地址和控制”的混合软件优化结构。

2.2 SIMD 寄存器与通用寄存器混合优化原理

传统标量程序每条指令通常只处理一个 32 bit 数据, 而 SIMD (Single Instruction Multiple Data, 单指令多数据) 允许一条指令同时对多个 32 bit 数据执行相同运算。SM3 的消息扩展和压缩函数包含大量结构相同的模 2^{32} 加法、异或、与、或和循环移位, 因此适合使用 SIMD 指令对多条独立消息进行并行处理。

由于同一条 SM3 消息的压缩轮次之间存在严格的数据依赖, 第 $j+1$ 轮必须使用第 j 轮产生的状态, 因此难以直接并行同一条消息的多个轮次。本实验采用多缓冲区并行方法, 将多条相互独立消息的相同状态字组织到同一个 SIMD 寄存器中。例如:

$$\mathbf{A} = (A_0, A_1, \dots, A_{N-1}), \quad (32)$$

$$\mathbf{W}_j = (W_{j,0}, W_{j,1}, \dots, W_{j,N-1}), \quad (33)$$

其中, A_i 表示第 i 条消息的状态字 A , $W_{j,i}$ 表示第 i 条消息的第 j 个扩展字, N 表示 SIMD 寄存器中能够容纳的 32 bit lane 数。

一条向量加法指令可以同时计算

$$\mathbf{A} + \mathbf{B} = (A_0 + B_0, A_1 + B_1, \dots, A_{N-1} + B_{N-1}), \quad (34)$$

向量异或、向量移位和向量布尔运算也采用相同方式执行。因此, 多条消息的消息扩展和压缩函数可以在不同 lane 中同步进行。

但是, 消息指针、消息长度、分组编号和循环次数等控制信息在不同消息之间可能并不相同, 不适合完全使用 SIMD 寄存器统一处理。因此, 本实验采用 SIMD 寄存器与通用寄存器混合的实现结构。

表 2: SIMD 寄存器与通用寄存器的职责划分

类型	保存内容	主要作用
SIMD 寄存器	状态字 $A \sim H$ 、消息扩展字 W_j 、 W'_j 、 $SS1$ 、 $SS2$ 、 $TT1$ 和 $TT2$	并行执行模加、异或、布尔函数和循环移位等规则运算
通用寄存器	消息地址、消息长度、分组编号、循环计数器、活跃 lane 位图和尾部消息数量	完成地址计算、循环控制、条件判断、消息调度和尾部处理
局部内存	扩展字数组、消息填充块和临时打包数据	保存无法全部驻留在寄存器中的中间结果

因此, 整个优化实现可以概括为: 通用寄存器负责控制流、地址和消息管理, SIMD 寄存器负责 SM3 消息扩展和压缩函数中的批量数据计算。这种混合设计既能发挥 SIMD 的数据并行能力, 又能处理不同消息长度和不规则内存访问等实际问题。

2.3 x86 架构下的 AVX2 与 AVX-512 原理

x86-64 处理器具有 RAX、RBX、RCX、RDX、RSI、RDI、R8 等通用寄存器, 同时提供用于 SIMD 运算的 XMM、YMM 和 ZMM 寄存器。

其中, AVX2 使用 256 bit 的 YMM 寄存器。由于 SM3 的基本运算单位是 32 bit 无符号整数, 一个 YMM 寄存器能够容纳

$$N_{\text{AVX2}} = \frac{256}{32} = 8 \quad (35)$$

个 32 bit lane, 因此 AVX2 实现可以同时处理 8 条独立消息。

AVX2 中的八路状态字可以表示为

$$\mathbf{A}_{\text{AVX2}} = (A_0, A_1, A_2, A_3, A_4, A_5, A_6, A_7). \quad (36)$$

SM3 中的异或、加法和循环左移分别可以映射为 AVX2 向量指令。典型指令包括:

- `vpxor`: 同时对 8 个 32 bit lane 执行按位异或;
- `vpaddq`: 同时执行 8 路模 2^{32} 加法;
- `vpsllq`: 同时执行 8 路逻辑左移;
- `vpsrlq`: 同时执行 8 路逻辑右移;
- `vpor`: 将左右移结果合并, 实现循环左移;
- `vpbblendvb`: 根据掩码选择原状态或候选状态。

例如, 32 bit 循环左移可以表示为

$$\text{ROTL}(X, n) = (X \ll n) \vee (X \gg (32 - n)). \quad (37)$$

在 AVX2 中, 该操作由 `vpsllq`、`vpsrlq` 和 `vpor` 组合完成, 并且一次同时作用于 8 条消息。

AVX-512 在 AVX2 的基础上将向量寄存器扩展为 512 bit 的 ZMM 寄存器。一个 ZMM 寄存器可以容纳

$$N_{\text{AVX512}} = \frac{512}{32} = 16 \quad (38)$$

个 32 bit lane, 理论上能够一次并行处理 16 条消息。

AVX-512 还提供掩码寄存器和更丰富的向量指令。例如, `vprold` 可以直接完成 32 bit 循环左移, `vpternlogd` 可以实现三输入布尔逻辑, 从而减少部分布尔函数所需的指令数量。本实验所在的 x86 处理器支持 AVX2, 但不支持 AVX-512F。因此, x86 平台以 AVX2 八路实现作为主要运行后端。AVX-512 代码仅完成编译与静态反汇编验证, 不参与本机正确性和性能测试。

在最终 x86 机器代码中, 通用寄存器负责消息地址、循环变量和分组编号, YMM 寄存器负责八路 SM3 状态运算。反汇编中同时出现通用寄存器和 YMM 向量寄存器, 可以证明最终实现确实采用了混合寄存器设计。

2.4 ARM64 架构下的 NEON 指令集原理

ARM64 架构使用 X0–X30 作为 64 bit 通用寄存器, 低 32 bit 记为 W0–W30, 还提供 V0–V31 共 32 个 128 bit 向量寄存器, 用于执行 NEON/Advanced SIMD 运算。

SM3 的基本数据类型为 32 bit 无符号整数, 因此一个 128 bit V 寄存器可以划分为 4 个 32 bit lane:

$$N_{\text{NEON}} = \frac{128}{32} = 4. \quad (39)$$

NEON 四路状态字可以表示为

$$\mathbf{A}_{\text{NEON}} = (A_0, A_1, A_2, A_3). \quad (40)$$

在 ARM64 汇编中, `v0.4s` 表示将 V0 寄存器解释为 4 个 32 bit 单精度元素, `v0.16b` 表示将同一个寄存器解释为 16 个 8 bit 元素。对于 SM3, 主要使用 `.4s` 形式执行四路 32 bit 运算。

NEON 实现中常见的向量指令包括:

- `eor`: 对 4 个 32 bit lane 同时执行异或;
- `add`: 执行四路模 2^{32} 加法;
- `ushl`: 按照向量中的移位量执行逻辑移位;
- `orr`: 合并左移与右移结果, 实现循环左移;
- `bsl`: 按照向量掩码选择两个候选结果;
- `movi`: 向 V 寄存器中填入立即数或掩码。

NEON 中的 32 bit 循环左移仍采用

$$\text{ROTL}(X, n) = (X \ll n) \vee (X \gg (32 - n)) \quad (41)$$

的形式实现。程序分别生成左移和右移结果，再使用向量按位或合并。最终汇编通常表现为 `ushl` 与 `orr` 等指令的组合。

在 ARM64 实现中，X/W 通用寄存器负责消息指针、长度、分组计数和循环控制，V 寄存器负责四路消息扩展和压缩函数。例如：

- `x1`、`x2` 等 X 寄存器用于保存消息地址；
- `w3`、`w4` 等 W 寄存器用于保存循环计数和分组编号；
- `v0.4s`、`v1.4s` 等 V 寄存器保存四条消息的状态字；
- `eor`、`add`、`ushl` 和 `orr` 完成四路并行计算。

因此，ARM64 版本同样采用“通用寄存器负责控制，NEON 寄存器负责数据计算”的混合结构。

3 实验设计与验证方法

3.1 总体实验设计

本实验采用“标量参考实现、架构优化后端、运行时自动分发和多层验证”的总体设计。

首先实现符合 SM3 标准的标量版本，用作正确性参考。随后分别实现 x86 AVX2 八路后端和 ARM64 NEON 四路后端。其中，SIMD 寄存器负责多条消息的状态字、消息扩展和压缩函数计算，通用寄存器负责消息地址、消息长度、分组编号、循环控制和活跃 lane 管理。

公共接口根据当前处理器架构和指令集支持情况自动选择后端。在 x86 平台上优先选择 AVX2，在 ARM64 平台上优先选择 NEON。对于不能组成完整 SIMD 批次的剩余消息，程序自动调用标量后端处理。

整个实验的验证过程按照以下顺序进行：

- (1) 使用标准测试向量验证标量 SM3 的基本正确性；
- (2) 使用边界长度和随机消息，将 SIMD 后端与标量实现进行差分比较；
- (3) 使用非向量宽度整数倍的消息数量验证自动分发和标量尾部处理；
- (4) 使用 OpenSSL SM3 进行文件级差分验证；

- (5) 对目标文件进行反汇编，确认编译器生成对应架构的 SIMD 指令；
- (6) 在多种消息长度下测试标量后端与 SIMD 后端的吞吐率和每字节耗时。

因此，本实验不仅检查最终摘要是否正确，还分别验证了算法实现、架构后端、自动分发、目标指令和性能提升。

3.2 正确性测试设计

正确性验证采用由浅入深的多层测试方法，具体测试内容如表 3 所示。

表 3: SM3 正确性测试设计

测试类型	测试对象	验证目的
标准向量测试	空消息、abc、abcd 重复 16 次	验证标量实现的填充、消息扩展、压缩函数和摘要输出
边界长度测试	55/56、63/64、119/120 字节等长度	验证单分组与多分组交界处的填充和长度编码
随机差分测试	随机生成不同长度和内容的消息	比较 SIMD 后端与标量参考实现的摘要是否完全一致
自动分发测试	每轮处理 37 条消息	验证消息数量不能被 8 或 4 整除时的标量尾部处理
OpenSSL 差分测试	128 个不同长度和内容的文件	将本工程输出与成熟密码库 OpenSSL SM3 进行比较

x86 AVX2 后端执行 101 轮边界长度和随机消息差分测试，每轮同时处理 8 条消息；ARM64 NEON 后端同样执行 101 轮测试，每轮同时处理 4 条消息。自动分发接口执行 51 轮测试，每轮包含 37 条消息。

选择 37 条消息是因为 37 既不是 8 的整数倍，也不是 4 的整数倍。在 AVX2 平台上，消息按照

$$37 = 8 + 8 + 8 + 8 + 5$$

(42)

进行处理；在 NEON 平台上按照

$$37 = 4 \times 9 + 1$$

(43)

进行处理。因此，该测试能够同时覆盖 SIMD 批量路径和标量尾部路径。

仅通过程序运行成功不能证明最终机器代码实际使用了 SIMD 指令。编译器可能因为编译选项或代码结构问题，将部分 intrinsic 转换为标量指令。因此，本实验使用 objdump 对目标文件进行反汇编，并检查关键寄存器和指令。

对于 x86 AVX2 后端, 重点检查:

- YMM 向量寄存器;
- `vpxor` 向量异或指令;
- `vpaddd` 向量模加指令;
- `vpslld` 和 `vpsrld` 向量移位指令;
- `vpor` 向量按位或指令。

对于 ARM64 NEON 后端, 重点检查:

- `v*.4s` 和 `v*.16b` 向量寄存器;
- `eor` 向量异或指令;
- `add` 向量加法指令;
- `ushl` 向量移位指令;
- `orr` 向量按位或指令;
- X/W 通用寄存器和 V 向量寄存器的混合使用。

反汇编结果同时用于验证 SIMD 指令生成情况和通用寄存器与 SIMD 寄存器的职责分工。

3.3 性能测试方法

性能测试程序固定处理 16 条等长消息, 分别测试以下消息长度:

$$64, 256, 1024, 4096, 16384, 1048576 \text{ 字节}. \quad (44)$$

程序根据消息长度自动调整循环次数, 使每组测试处理的总数据量约为 256 MiB:

$$iterations \approx \frac{256 \times 2^{20}}{16 \times message_bytes}. \quad (45)$$

每种后端独立执行 7 次采样, 并使用中位数作为最终结果, 以降低偶然调度和系统负载造成的影响。性能指标包括:

- **MiB/s**: 每秒能够处理的数据量, 数值越大越好;
- **ns/byte**: 处理一个字节所需的时间, 数值越小越好;

- **speedup**: SIMD 后端吞吐率与同平台标量后端吞吐率之比。

测试时使用

```
taskset -c 0
```

将程序固定到逻辑 CPU 0，减少进程在不同核心之间迁移造成的性能波动。

由于 x86 和 ARM64 实验运行于不同处理器和不同系统环境，本实验只比较各自平台内部的 SIMD 后端与标量后端，不直接使用绝对吞吐率比较两种架构。

4 x86 AVX2 实现与实验

4.1 AVX2 八路实现

AVX2 后端使用 `__m256i` 表示 256 bit YMM 寄存器。八条消息的同一状态字组成一个向量。例如：

$$\mathbf{A} = (A_0, A_1, \dots, A_7). \quad (46)$$

32 bit 循环左移由逻辑左移、逻辑右移和按位或组合实现：

Listing 1: AVX2 32 bit 循环左移

```
1 static inline __m256i v256_rotl32(__m256i x, int n)
2 {
3     return _mm256_or_si256(
4         _mm256_slli_epi32(x, n),
5         _mm256_srli_epi32(x, 32 - n));
6 }
```

向量版 P_1 可直接由 AVX2 异或和循环移位 intrinsic 组成：

Listing 2: AVX2 向量版 P_1

```
1 static inline __m256i v256_p1(__m256i x)
2 {
3     return _mm256_xor_si256(
4         x,
5         _mm256_xor_si256(
6             v256_rotl32(x, 15),
7             v256_rotl32(x, 23)));
8 }
```

不同长度消息使用活跃位图构造向量掩码，并在压缩完成后选择性写回：

Listing 3: AVX2 活跃 lane 选择性写回

```
1 const __m256i mask = v256_active_mask(active_bits);
2
3 for (unsigned i = 0; i < 8u; ++i) {
4     state[i] =
5         _mm256_blendv_epi8(state[i], candidate[i], mask);
6 }
```

4.2 x86 实验流程

编译和正确性测试命令如下:

Listing 4: x86 编译与正确性测试

```
1 make clean
2 make -j"$(nproc)" all
3 make test | tee results/x86-correctness.txt
```

使用 objdump 检查 AVX2 指令:

Listing 5: AVX2 反汇编检查

```
1 objdump -d -Mintel build/sm3_avx2.o \
2     > results/sm3_avx2.asm
3
4 grep -Ei \
5 'ymm|vpadd|vpxor|vpsll|vpsrl|vpblend|vpor' \
6 results/sm3_avx2.asm | head -n 30
```

性能测试覆盖 64 B、256 B、1024 B、4096 B、16384 B 和 1 MiB:

Listing 6: x86 多长度性能测试

```
1 for size in 64 256 1024 4096 16384 1048576
2 do
3     taskset -c 0 ./build/sm3_bench "$size" 256 \
4         | tee "results/local-x86/bench-${size}.txt"
5 done
```

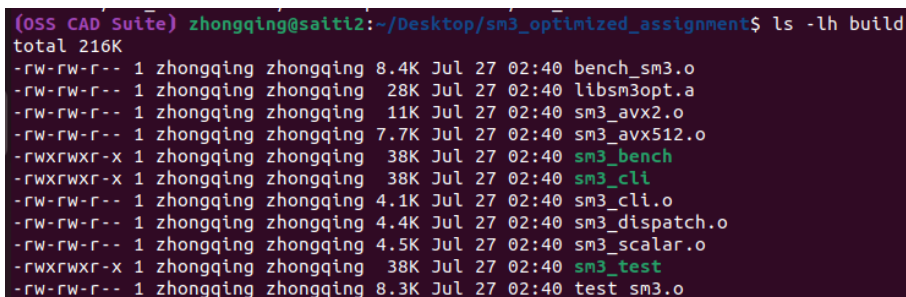
4.3 x86 正确性与指令验证

x86 测试输出为:

```
[PASS] scalar vector empty
```

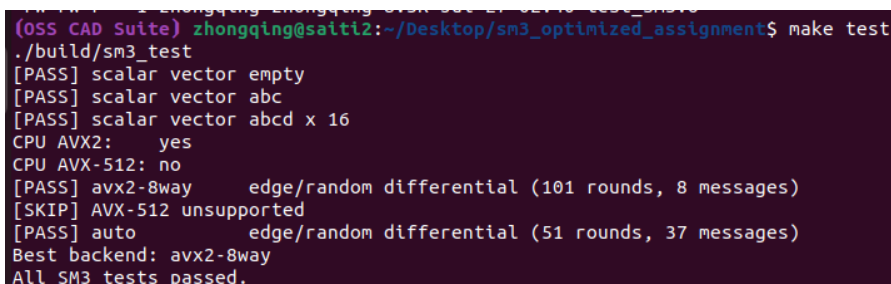
```
[PASS] scalar vector abc
[PASS] scalar vector abcd x 16
CPU AVX2: yes
CPU AVX-512: no
[PASS] avx2-8way edge/random differential
        (101 rounds, 8 messages)
[SKIP] AVX-512 unsupported
[PASS] auto edge/random differential
        (51 rounds, 37 messages)
Best backend: avx2-8way
All SM3 tests passed.
```

三个标准向量均通过, 说明标量参考实现正确。AVX2 八路实现通过 101 轮边界长度和随机消息差分测试, 每轮同时处理 8 条消息, 所得摘要与标量实现完全一致。自动分发接口也通过了 37 条消息测试, 并选择 avx2-8way 作为最佳后端。



```
(OSS CAD Suite) zhongqing@sait12:~/Desktop/sm3_optimized_assignment$ ls -lh build
total 216K
-rw-rw-r-- 1 zhongqing zhongqing 8.4K Jul 27 02:40 bench_sm3.o
-rw-rw-r-- 1 zhongqing zhongqing 28K Jul 27 02:40 libsm3opt.a
-rw-rw-r-- 1 zhongqing zhongqing 11K Jul 27 02:40 sm3_avx2.o
-rw-rw-r-- 1 zhongqing zhongqing 7.7K Jul 27 02:40 sm3_avx512.o
-rwxrwxr-x 1 zhongqing zhongqing 38K Jul 27 02:40 sm3_bench
-rwxrwxr-x 1 zhongqing zhongqing 38K Jul 27 02:40 sm3_cli
-rw-rw-r-- 1 zhongqing zhongqing 4.1K Jul 27 02:40 sm3_cli.o
-rw-rw-r-- 1 zhongqing zhongqing 4.4K Jul 27 02:40 sm3_dispatch.o
-rw-rw-r-- 1 zhongqing zhongqing 4.5K Jul 27 02:40 sm3_scalar.o
-rwxrwxr-x 1 zhongqing zhongqing 38K Jul 27 02:40 sm3_test
-rw-rw-r-- 1 zhongqing zhongqing 8.3K Jul 27 02:40 test_sm3.o
```

图 2: x86 平台 AVX2 指令集检测结果



```
(OSS CAD Suite) zhongqing@sait12:~/Desktop/sm3_optimized_assignment$ make test
./build/sm3_test
[PASS] scalar vector empty
[PASS] scalar vector abc
[PASS] scalar vector abcd x 16
CPU AVX2: yes
CPU AVX-512: no
[PASS] avx2-8way edge/random differential (101 rounds, 8 messages)
[SKIP] AVX-512 unsupported
[PASS] auto edge/random differential (51 rounds, 37 messages)
Best backend: avx2-8way
All SM3 tests passed.
```

图 3: x86 AVX2 正确性与自动分发测试结果

反汇编结果中出现 ymm0--ymm15、vpxor、vpslld、vpsrld 和 vpor。其中 vpxor 用于消息扩展和压缩函数中的向量异或, vpslld、vpsrld 与 vpor 共同构造循环左移。该结果证明编译器生成了真实 AVX2 机器指令, 而不是将 intrinsic 退化为普通标量代码。

```

(OSS CAD Suite) zhongqing@sait12:~/Desktop/sm3_optimized_assignment$ make asm-check
objdump -d -Mintel build/sm3_avx2.o > build/sm3_avx2.asm
objdump -d -Mintel build/sm3_avx512.o > build/sm3_avx512.asm
AVX2 evidence:
   a8:    c5 fd 6f ac 24 c0 00    vmovdqa ymm5,YMMWORD PTR [rsp+0xc0]
   b1:    c5 fd 7f 2c ca        vmovdqa YMMWORD PTR [rdx+rcx*8],ymm5
  179:    c5 fd 6f 9c 24 80 02    vmovdqa ymm3,YMMWORD PTR [rsp+0x280]
 187:    c5 fd 6f 94 24 a0 02    vmovdqa ymm2,YMMWORD PTR [rsp+0x2a0]
 198:    c5 fd 6f 84 24 c0 02    vmovdqa ymm0,YMMWORD PTR [rsp+0x2c0]
 1a1:    c5 7d 6f 8c 24 c0 01    vmovdqa ymm9,YMMWORD PTR [rsp+0x1c0]
 1aa:    c5 7d 6f 84 24 e0 01    vmovdqa ymm8,YMMWORD PTR [rsp+0x1e0]
 1b3:    c5 fd 6f bc 24 00 02    vmovdqa ymm7,YMMWORD PTR [rsp+0x200]
 1bc:    c5 fd 6f b4 24 e0 00    vmovdqa ymm6,YMMWORD PTR [rsp+0xe0]
 1c5:    c5 fd 6f ac 24 00 01    vmovdqa ymm5,YMMWORD PTR [rsp+0x100]
 1ce:    c5 fd 6f a4 24 20 01    vmovdqa ymm4,YMMWORD PTR [rsp+0x120]
 1d7:    c5 ad 72 d3 11        vpsrlq ymm10,ymm3,0x11
 1dc:    c4 c1 4d ef c9        vpxor  ymm1,ymm6,ymm9
 1e1:    c5 fd 6f 31          vmovdqa ymm6,YMMWORD PTR [rcx]
 1e8:    c5 e5 72 f3 0f        vpsllq ymm3,ymm3,0xf
 1ed:    c5 7d 6f 89 e0 00 00    vmovdqa ymm9,YMMWORD PTR [rcx+0xe0]
AVX-512 evidence:
 115:    62 f1 fd 48 6f 5c 24    vmovdqa64 zmm3,ZMMWORD PTR [rsp+0x40]
 125:    62 f1 fd 48 7f 59 ff    vmovdqa64 ZMMWORD PTR [rcx-0x40],zmm3
 286:    62 f1 fd 48 6f 5c 24    vmovdqa64 zmm3,ZMMWORD PTR [rsp+0x3c0]
 293:    62 f1 fd 48 6f 54 24    vmovdqa64 zmm2,ZMMWORD PTR [rsp+0x400]
 2a3:    62 f1 fd 48 6f 4c 24    vmovdqa64 zmm1,ZMMWORD PTR [rsp+0x440]
 2ab:    62 71 fd 48 6f 54 24    vmovdqa64 zmm10,ZMMWORD PTR [rsp+0x240]
 2b3:    62 71 fd 48 6f 4c 24    vmovdqa64 zmm9,ZMMWORD PTR [rsp+0x280]
 2bb:    62 71 fd 48 6f 44 24    vmovdqa64 zmm8,ZMMWORD PTR [rsp+0x2c0]
 2c3:    62 f1 fd 48 6f 7c 24    vmovdqa64 zmm7,ZMMWORD PTR [rsp+0x80]
 2cb:    62 f1 fd 48 6f 74 24    vmovdqa64 zmm6,ZMMWORD PTR [rsp+0xc0]
 2d3:    62 f1 fd 48 6f 64 24    vmovdqa64 zmm4,ZMMWORD PTR [rsp+0x100]
 2db:    62 f1 65 48 72 cb 0f    vprold  zmm3,zmm3,0xf
 2e2:    62 f1 7d 48 6f c7        vmovdqa32 zmm0,zmm7
 2e8:    62 f1 fd 48 6f 3a        vmovdqa64 zmm7,ZMMWORD PTR [rdx]
 2f1:    62 f1 6d 48 72 ca 0f    vprold  zmm2,zmm2,0xf
 2ff:    62 f1 75 48 72 c9 0f    vprold  zmm1,zmm1,0xf

```

图 4: x86 AVX2 目标指令反汇编结果

AVX-512 目标文件中还出现了 `zmm`、`vprold` 和 `vpternlogd`。由于当前处理器不支持 AVX-512F，该部分只证明 AVX-512 代码能够成功生成，不作为本机运行结果。

4.4 x86 性能结果

性能程序每次处理 16 条等长消息。不同消息长度下，程序自动调整循环次数，使每组测试总处理量约为 256 MiB；每个后端独立采样 7 次并取中位数。

表 4: x86 标量与 AVX2 八路性能结果

消息长度/B	Scalar/MiB/s	AVX2/MiB/s	Scalar/ns/B	AVX2/ns/B	加速比
64	65.594	379.183	14.5390	2.5151	5.781
256	112.485	794.940	8.4782	1.1997	7.067
1024	160.773	961.262	5.9318	0.9921	5.979
4096	167.655	1017.552	5.6883	0.9372	6.069
16384	167.266	1034.755	5.7015	0.9216	6.186
1048576	172.510	1024.854	5.5282	0.9305	5.941

```
(OSS CAD Suite) zhongqing@sait12:~/Desktop/sm3_optimized_assignment$ for size in 64 256 1024 4096 16384 1048576
do
    echo "==== message size: ${size} bytes ====="
    taskset -c 0 ./build/sm3_bench "$size" 256 \
        | tee "results/local-x86/bench-${size}.txt"
done
==== message size: 64 bytes =====
SM3 multi-buffer benchmark
message_bytes=64 messages=16 iterations=262144 samples=7
backend      MiB/s      ns/byte      speedup
scalar       65.594      14.5390      1.000
avx2-8way    379.183     2.5151       5.781
sink=0
==== message size: 256 bytes =====
SM3 multi-buffer benchmark
message_bytes=256 messages=16 iterations=65536 samples=7
backend      MiB/s      ns/byte      speedup
scalar       112.485     8.4782       1.000
avx2-8way    794.940     1.1997       7.067
sink=0
==== message size: 1024 bytes =====
SM3 multi-buffer benchmark
message_bytes=1024 messages=16 iterations=16384 samples=7
backend      MiB/s      ns/byte      speedup
scalar       160.773     5.9318       1.000
avx2-8way    961.262     0.9921       5.979
sink=0
==== message size: 4096 bytes =====
SM3 multi-buffer benchmark
message_bytes=4096 messages=16 iterations=4096 samples=7
backend      MiB/s      ns/byte      speedup
scalar       167.655     5.6883       1.000
avx2-8way    1017.552    0.9372       6.069
sink=0
==== message size: 16384 bytes =====
SM3 multi-buffer benchmark
message_bytes=16384 messages=16 iterations=1024 samples=7
backend      MiB/s      ns/byte      speedup
scalar       167.266     5.7015       1.000
avx2-8way    1034.755    0.9216       6.186
sink=0
==== message size: 1048576 bytes =====
SM3 multi-buffer benchmark
message_bytes=1048576 messages=16 iterations=16 samples=7
backend      MiB/s      ns/byte      speedup
scalar       172.510     5.5282       1.000
avx2-8way    1024.854    0.9305       5.941
sink=0
```

图 5: x86 标量与 AVX2 多长度性能测试

AVX2 在六种消息长度下的加速比为 5.781 ~ 7.067, 平均加速比为

$$\frac{5.781 + 7.067 + 5.979 + 6.069 + 6.186 + 5.941}{6} = 6.171. \quad (47)$$

64 B 消息中, AVX2 吞吐率由 65.594 MiB/s 提高到 379.183 MiB/s; 4096 B 时达到 1017.552 MiB/s; 1 MiB 时达到 1024.854 MiB/s。

随着消息长度增加, 初始化、消息填充、函数调用和数据打包等固定开销逐渐被摊薄, 因此吞吐率趋于稳定。AVX2 理论上具有 8 个 32 bit lane, 但实际加速低于 8 倍, 主要原因包括输入打包、扩展字内存访问、活跃掩码生成以及 SM3 轮函数的数据依赖。约 6.17 倍的平均加速说明 AVX2 八路并行能够显著提高 SM3 批量吞吐率。

5 ARM64 NEON 实现与实验

5.1 NEON 四路实现

ARM64 NEON 使用 `uint32x4_t` 表示 128 bit V 寄存器。四条消息的同一状态字组成一个向量:

$$\mathbf{A} = (A_0, A_1, A_2, A_3). \quad (48)$$

循环左移使用 `vshlq_u32` 的带符号移位计数实现。正表示左移, 负表示右移:

Listing 7: NEON 32 bit 循环左移

```

1 static inline uint32x4_t
2 v128_rotl32(uint32x4_t x, int n)
3 {
4     const int32x4_t left  = vdupq_n_s32(n);
5     const int32x4_t right = vdupq_n_s32(n - 32);
6
7     return vorrq_u32(
8         vshlq_u32(x, left),
9         vshlq_u32(x, right));
10 }
```

候选链值通过 `vbslq_u32` 按活跃掩码选择性写回:

Listing 8: NEON 活跃 lane 选择性写回

```

1 const uint32x4_t mask =
2     v128_active_mask(active_bits);
3
4 for (unsigned i = 0; i < 8u; ++i) {
5     state[i] =
6         vbslq_u32(mask, candidate[i], state[i]);
7 }
```

5.2 ARM64 实验流程

实验使用阿里云 ECS 计算型 c8y ARM64 实例。登录服务器后首先验证体系结构和 NEON 特性:

Listing 9: ARM64 环境检测

```

1 uname -m
```

```
2 grep -m1 '^Features' /proc/cpuinfo
```

```
Last login: Mon Jul 27 15:58:42 2026 from 100.104.86.48
root@iZbp1ix16156fwdx8kc44Z:~# uname -m
aarch64
root@iZbp1ix16156fwdx8kc44Z:~# grep -m1 '^Features' /proc/cpuinfo
Features          : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt fcm
sm4 asimddp sha512 sve asimdfeh dit uscat ilrcpc flagm sb dcpodp sve2 sveaes svepmull svebitperm svesha3 svesm4 flagm2 frint svei
8mm svebf16 i8mm bf16 dgh bti
```

图 6: ARM64

输出中 `uname -m` 为 `aarch64`, `Features` 中包含 `asimd`。

原生编译和测试命令如下:

Listing 10: ARM64 原生编译与测试

```
1 make clean
2 make -j"$(nproc)" all
3
4 file build/sm3_test
5 file build/sm3_neon.o
6
7 make test \
8 | tee results/local-arm64/correctness.txt
```

与 OpenSSL 的差分验证命令如下:

Listing 11: ARM64 OpenSSL 差分验证

```
1 chmod +x scripts/*
2 make verify-openssl \
3 | tee results/local-arm64/openssl-differential.txt
```

生成 NEON 反汇编:

Listing 12: ARM64 NEON 反汇编

```
1 objdump -d build/sm3_neon.o \
2 > results/local-arm64/sm3_neon.asm
3
4 grep -Ei \
5 'v[0-9]+|\.4s|\.16b|eor|add|ushl|orr|bsl' \
6 results/local-arm64/sm3_neon.asm | head -n 50
```

性能测试命令如下:

Listing 13: ARM64 多长度性能测试

```
1 for size in 64 256 1024 4096 16384 1048576
2 do
```

```
3 taskset -c 0 ./build/sm3_bench "$size" 256 \  
4 | tee "results/local-arm64/bench-${size}.txt"  
5 done
```

5.3 ARM64 正确性与指令验证

ARM64 目标文件检查结果显示:

```
build/sm3_test:  
ELF 64-bit LSB executable, ARM aarch64  
  
build/sm3_neon.o:  
ELF 64-bit LSB relocatable, ARM aarch64
```

表明程序是在真实 ARM64 平台上原生编译,而不是在 x86 平台上进行交叉编译。

正确性测试输出为:

```
[PASS] scalar vector empty  
[PASS] scalar vector abc  
[PASS] scalar vector abcd x 16  
CPU NEON: yes  
[PASS] neon-4way edge/random differential  
        (101 rounds, 4 messages)  
[PASS] auto edge/random differential  
        (51 rounds, 37 messages)  
Best backend: neon-4way  
All SM3 tests passed.
```

NEON 四路实现完成 101 轮边界长度和随机消息差分测试,每轮处理 4 条消息,所有摘要均与标量参考实现一致。自动分发接口处理了 37 条消息,并自动选择 neon-4way。

```
root@iZbp1xl6156fwdx8kc44Z:~/sm3_optimized_assignment# mkdir -p results/local-arm64  
  
make test | tee results/local-arm64/correctness.txt  
./build/sm3_test  
[PASS] scalar vector empty  
[PASS] scalar vector abc  
[PASS] scalar vector abcd x 16  
CPU NEON: yes  
[PASS] neon-4way      edge/random differential (101 rounds, 4 messages)  
[PASS] auto          edge/random differential (51 rounds, 37 messages)  
Best backend: neon-4way  
All SM3 tests passed. 🧠
```

图 7: ARM64 NEON 正确性与自动分发测试

OpenSSL 差分测试生成了 128 个不同长度和内容的文件,分别使用本工程自动分发接口和 OpenSSL SM3 计算摘要,结果为:

```
[PASS] 128 files matched OpenSSL SM3; backend auto-dispatch exercised
root@izbp1ixl6156fwdx8kc44Z:~/sm3_optimized_assignment# python3 scripts/verify_openssl.py \
| tee results/local-arm64/openssl-differential.txt
[PASS] 128 files matched OpenSSL SM3; backend auto-dispatch exercised
root@izbp1ixl6156fwdx8kc44Z:~/sm3_optimized_assignment#
```

图 8: ARM64 实现与 OpenSSL SM3 差分验证

```
[PASS] 128 files matched OpenSSL SM3;
backend auto-dispatch exercised
```

这说明 ARM64 实现能够正确处理不同长度、填充边界和多分组文件输入。

ARM64 反汇编中同时出现 `x1`、`w3`、`sp` 等通用寄存器和 `v0.4s`、`v10.16b` 等 NEON 寄存器。`ushl` 用于四路移位，`orr` 用于组合左右移结果，`eor` 用于消息扩展和压缩函数中的异或。该结果直接证明最终机器码采用了“通用寄存器负责地址和控制，NEON 寄存器负责并行计算”的混合结构。

```
root@izbp1ixl6156fwdx8kc44Z:~/sm3_optimized_assignment# grep -Ei \
'v[0-9]+|\.4s|\.16b|eor|add|shl|ushr|ushl|orr|bsl|sri' \
results/local-arm64/sm3_neon.asm \
| head -n 50 \
| tee results/local-arm64/disassembly-summary.txt
18c: 910343e1 add x1, sp, #0xd0
190: 4f0005e1 movi v1.4s, #0xf
198: 6f000600 mvni v0.4s, #0x10
19c: 4f0006e6 movi v6.4s, #0x17
1a0: 6f00050d mvni v13.4s, #0x8
1a4: 4f0004e5 movi v5.4s, #0x7
1a8: 6f000708 mvni v8.4s, #0x18
1cc: 4eb71ef4 mov v20.16b, v23.16b
1d0: 6ea0449b ushl v27.4s, v4.4s, v0.4s
1d4: 11001863 add w3, w3, #0x6
1d8: 6ea1448a ushl v10.4s, v4.4s, v1.4s
1dc: 6ea04478 ushl v24.4s, v3.4s, v0.4s
1e0: 4eb31e77 mov v23.16b, v19.16b
1e4: 4eb21e4b mov v11.16b, v18.16b
1e8: 6ea14479 ushl v25.4s, v3.4s, v1.4s
1ec: 4ebb1d4a orr v10.16b, v10.16b, v27.16b
1f4: 6ea14456 ushl v22.4s, v2.4s, v1.4s
1f8: 6ea04447 ushl v7.4s, v2.4s, v0.4s
1fc: 4eb81f39 orr v25.16b, v25.16b, v24.16b
200: 6e331d4a eor v10.16b, v10.16b, v19.16b
204: 6ea8463a ushl v26.4s, v17.4s, v8.4s
208: 4eb11e38 mov v24.16b, v17.16b
20c: 4eb61ce7 orr v7.16b, v7.16b, v22.16b
210: 6e291d49 eor v9.16b, v10.16b, v9.16b
214: 6ea54636 ushl v22.4s, v17.4s, v5.4s
218: 6e321f39 eor v25.16b, v25.16b, v18.16b
220: 4eba1ed6 orr v22.16b, v22.16b, v26.16b
224: 6e371f39 eor v25.16b, v25.16b, v23.16b

224: 6e371f39 eor v25.16b, v25.16b, v23.16b
228: 6ea0453c ushl v28.4s, v9.4s, v0.4s
22c: 6ead453d ushl v29.4s, v9.4s, v13.4s
230: 6ea1453a ushl v26.4s, v9.4s, v1.4s
234: 6ea6452a ushl v10.4s, v9.4s, v6.4s
238: 6e311ce7 eor v7.16b, v7.16b, v17.16b
23c: 6ea04737 ushl v23.4s, v25.4s, v0.4s
240: 4ebc1f5a orr v26.16b, v26.16b, v28.16b
244: 4ebd1d4a orr v10.16b, v10.16b, v29.16b
248: 6e2b1ce7 eor v7.16b, v7.16b, v11.16b
24c: 6ead473d ushl v29.4s, v25.4s, v13.4s
250: 6ea6473c ushl v28.4s, v25.4s, v6.4s
254: 6ea1472b ushl v11.4s, v25.4s, v1.4s
258: 6e2a1f5a eor v26.16b, v26.16b, v10.16b
25c: 6ead44ea ushl v10.4s, v7.4s, v13.4s
260: 4ebd1f9c orr v28.16b, v28.16b, v29.16b
264: 6ea644fd ushl v29.4s, v7.4s, v6.4s
268: 6ea5461b ushl v27.4s, v16.4s, v5.4s
26c: 4eb71d6b orr v11.16b, v11.16b, v23.16b
270: 6ea144fe ushl v30.4s, v7.4s, v1.4s
274: 6ea044f7 ushl v23.4s, v7.4s, v0.4s
278: 4ebd1d4a orr v10.16b, v10.16b, v29.16b
27c: 6ea8461d ushl v29.4s, v16.4s, v8.4s
```

图 9: ARM64 实现中通用寄存器与 NEON 向量寄存器的混合使用

5.4 ARM64 性能结果

表 5: ARM64 标量与 NEON 四路性能结果

消息长度/B	Scalar/MiB/s	NEON/MiB/s	Scalar/ns/B	NEON/ns/B	加速比
64	65.948	130.392	14.4611	7.3139	1.977
256	107.075	215.166	8.9066	4.4323	2.009
1024	126.846	257.520	7.5184	3.7033	2.030
4096	132.358	270.298	7.2053	3.5282	2.042
16384	134.649	273.706	7.0827	3.4843	2.033
1048576	135.068	274.546	7.0607	3.4736	2.033

```

root@izbp1ix16156fwdx8kc44Z:~/sm3_optimized_assignment# for size in 64 256 1024 4096 16384 1048576
do
    echo "==== message size: ${size} bytes ====="
    taskset -c 0 ./build/sm3_bench "${size}" 256 \
    | tee "results/local-arm64/bench-${size}.txt"
done
==== message size: 64 bytes =====
SM3 multi-buffer benchmark
message_bytes=64 messages=16 iterations=262144 samples=7
backend      MiB/s      ns/byte      speedup
scalar       65.948      14.4611      1.000
neon-4way    130.392     7.3139       1.977
sink=0
==== message size: 256 bytes =====
SM3 multi-buffer benchmark
message_bytes=256 messages=16 iterations=65536 samples=7
backend      MiB/s      ns/byte      speedup
scalar       107.075     8.9066       1.000
neon-4way    215.166     4.4323       2.009
sink=0
==== message size: 1024 bytes =====
SM3 multi-buffer benchmark
message_bytes=1024 messages=16 iterations=16384 samples=7
backend      MiB/s      ns/byte      speedup
scalar       126.846     7.5184       1.000
neon-4way    257.520     3.7033       2.030
sink=0
==== message size: 4096 bytes =====
SM3 multi-buffer benchmark
message_bytes=4096 messages=16 iterations=4096 samples=7
backend      MiB/s      ns/byte      speedup
scalar       132.358     7.2053       1.000
neon-4way    270.298     3.5282       2.042
sink=0
==== message size: 16384 bytes =====
SM3 multi-buffer benchmark
message_bytes=16384 messages=16 iterations=1024 samples=7
backend      MiB/s      ns/byte      speedup
scalar       134.649     7.0827       1.000
neon-4way    273.706     3.4843       2.033
sink=0
==== message size: 16384 bytes =====
SM3 multi-buffer benchmark
message_bytes=16384 messages=16 iterations=1024 samples=7
backend      MiB/s      ns/byte      speedup
scalar       134.649     7.0827       1.000
neon-4way    273.706     3.4843       2.033
sink=0
==== message size: 1048576 bytes =====
SM3 multi-buffer benchmark
message_bytes=1048576 messages=16 iterations=16 samples=7
backend      MiB/s      ns/byte      speedup
scalar       135.068     7.0607       1.000
neon-4way    274.546     3.4736       2.033
sink=0
root@izbp1ix16156fwdx8kc44Z:~/sm3_optimized_assignment#

```

图 10: ARM64 标量与 NEON 多长度性能测试

NEON 在六种消息长度下的加速比为 1.977 ~ 2.042, 平均加速比为

$$\frac{1.977 + 2.009 + 2.030 + 2.042 + 2.033 + 2.033}{6} = 2.021. \quad (49)$$

64 B 时, NEON 吞吐率由 65.948 MiB/s 提高至 130.392 MiB/s; 4096 B 时达到 270.298 MiB/s; 1 MiB 时达到 274.546 MiB/s。4096 B 以后吞吐率基本稳定在 270–275 MiB/s, 每字节耗时约为 3.5 ns。

NEON 理论上具有 4 个 32 bit lane, 但实际加速约为 2 倍。主要原因是消息指针、长度、尾块构造和活跃位图仍由标量代码处理, 向量装载与写回也会引入额外开销, 同时 SM3 压缩函数存在较长数据依赖链。云服务器虚拟化调度也可能带来一定性能波动。稳定的约 2.02 倍加速说明 NEON 四路实现能够有效提高批量 SM3 处理能力。

6 实验结果分析

6.1 x86 与 ARM64 结果对比

表 6: 两种架构优化效果汇总

平台	SIMD 宽度	加速范围	平均加速比
x86 AVX2	8 个 32 bit lane	5.781–7.067	6.171
ARM64 NEON	4 个 32 bit lane	1.977–2.042	2.021

x86 与 ARM64 数据来自不同处理器和运行环境, 因此不能直接使用绝对吞吐率判断 AVX2 与 NEON 哪一种“更快”。合理的比较方式是在同一平台内部计算 SIMD 实现相对于标量实现的加速比。

AVX2 的平均加速约为 6.17 倍, 接近但低于理论 8 路宽度; NEON 的平均加速约为 2.02 倍, 低于理论 4 路宽度。说明实际性能除了受到 SIMD 宽度影响, 还取决于指令吞吐率、寄存器数量、数据打包、内存访问、轮函数依赖和虚拟化环境。

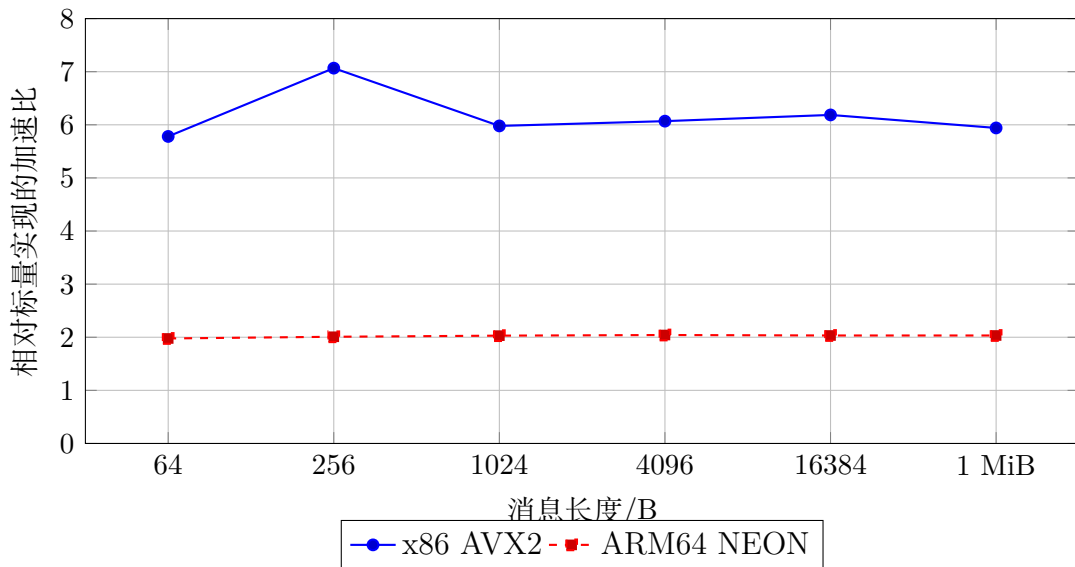


图 11: 不同消息长度下 AVX2 与 NEON 相对标量实现的加速比

由图 11 可以看出, x86 AVX2 和 ARM64 NEON 在不同消息长度下均能取得稳定的性能提升。AVX2 的加速比总体保持在约 6 倍, 其中 256 B 消息下达到最高的 7.067 倍; NEON 的加速比稳定在约 2 倍, 并在 4096 B 消息下达到 2.042 倍。

随着消息长度增加, 两种 SIMD 后端的加速比均逐渐趋于稳定, 说明初始化、消息填充、函数调用和数据打包等固定开销在长消息中被逐步摊薄, 性能主要由消息扩展和压缩函数的向量化效率决定。

需要说明的是, x86 与 ARM64 数据来自不同处理器和运行环境, 该图仅用于展示两种优化方案各自相对于同平台标量实现的加速趋势, 不能用于直接比较两种处理器的绝对性能。

6.2 实验中遇到的问题

x86 不支持 AVX-512。 本机仅支持 AVX2, 因此 AVX-512 测试显示 [SKIP] AVX-512 unsupported。该结果不是程序错误。题目要求 x86 端实现 AVX2 或 AVX-512 中的一种, 因此采用 AVX2 即可满足要求。

ARM64 服务器选择。 普通经济型和 Intel 实例属于 x86 架构, 无法运行 NEON。最终选择阿里云 ARM 计算型 c8y 实例。

脚本执行权限丢失。 RAR 压缩包从 Windows 上传到 Linux 后, verify_openssl.py 失去可执行权限, 运行时出现 Permission denied。执行

```
1 chmod +x scripts/*
```

后验证成功。也可以直接使用

```
1 python3 scripts/verify_openssl.py
```

运行脚本。

6.3 进一步优化方向

本实现仍可从以下方面进一步改进：

- (1) 将消息扩展与轮函数交错执行，减少 68 个向量扩展字带来的栈访问；
- (2) 针对固定长度消息优化 AoS 到 SoA 的打包和转置；
- (3) 对压缩轮函数进行适度展开，减少循环控制和状态搬移；
- (4) 使用 `perf stat` 采集 cycles、instructions 和 cache-misses；
- (5) 在支持 AVX-512 的真实处理器上补充十六路实现的原生测试；
- (6) 在 ARM64 平台进一步比较通用 NEON 实现与 ARM 专用 SM3 指令扩展。

7 结论

本实验完成了 SM3 标量参考实现、x86 AVX2 八路并行实现和 ARM64 NEON 四路并行实现，满足“实现基于 SIMD 寄存器和通用寄存器混合的 SM3 算法优化，并支持 ARM64 和 x86 两种架构指令集”的作业要求。

在实现结构上，YMM 或 V 向量寄存器负责状态字、消息扩展、布尔函数、模加和循环移位；通用寄存器负责消息指针、长度、分组编号、循环控制、活跃 lane 位图和尾部分组管理。不同长度消息通过向量掩码选择性写回，不能组成完整向量批次的剩余消息由标量后端处理。

在正确性方面，AVX2 和 NEON 均通过标准测试向量、边界长度和随机消息差分测试。ARM64 版本的 128 个文件摘要全部与 OpenSSL SM3 一致。反汇编结果证明 x86 版本生成了 YMM/AVX2 指令，ARM64 版本同时使用 X/W 通用寄存器和 V/NEON 指令。

在性能方面，AVX2 在六种消息长度下平均取得约 6.171 倍批量吞吐率加速，NEON 平均取得约 2.021 倍加速。实验结果表明，多缓冲区 SIMD 与通用寄存器混合设计能够在保证算法正确性的同时显著提高 SM3 的批量处理性能。

参考文献

- [1] 国家质量监督检验检疫总局、国家标准化管理委员会. GB/T 32905-2016 信息安全技术 SM3 密码杂凑算法, 2016.
- [2] Intel Corporation. Intel 64 and IA-32 Architectures Software Developer's Manual; Intel Intrinsics Guide.
- [3] Arm Limited. Arm Neon Intrinsics Reference; Arm Architecture Reference Manual for A-profile Architecture.
- [4] OpenSSL Project. OpenSSL command-line digest implementation and SM3 provider support.